Report of DDPG for Continous Control Problem (Udacity DRLND)

Jun-Jie Zhang@USTC

1. Implementation and model architectures

We mainly follow the algorithm of DDPG with experience replay:

Algorithm 1 Deep Deterministic Policy Gradient

Input: initial policy parameters θ, Q-function parameters ϕ, empty replay buffer $\mathcal{D}$
Set target parameters equal to main parameters $\theta_{\text{targ}} \leftarrow \theta$, $\phi_{\text{targ}} \leftarrow \phi$
repeat
  Observe state s and select action $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{Low}, a_{High})$, where $\epsilon \sim \mathcal{N}$
  Execute a in the environment
  Observe next state s', reward r, and done signal d to indicate whether s' is terminal
  Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
  If s' is terminal, reset environment state.
  if it's time to update **then**
    for however many updates **do**
      Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
      Compute targets

$$y(r, s', d) = r + \gamma(1 - d) Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s'))$$

      Update Q-function by one step of gradient descent using

$$\nabla_\phi \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left(Q_\phi(s, a) - y(r, s', d)\right)^2$$

      Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} Q_\phi(s, \mu_\theta(s))$$

      Update target networks with

$$\phi_{\text{targ}} \leftarrow \rho \phi_{\text{targ}} + (1 - \rho)\phi$$
$$\theta_{\text{targ}} \leftarrow \rho \theta_{\text{targ}} + (1 - \rho)\theta$$

    end for
  end if
until convergence

<u>Architecture of the *Actor* net.</u>

We use three-layer fully connected nets as a surrogate of the Actor. Each layer (except for the last layer) is applied a leaky relu activation function,

and batch normalization is applied after the first layer. This net is sketched as follows:

```
        self.fc1 = nn.Linear(state_size, fc1_units)
        self.bn1 = nn.BatchNorm1d(fc1_units)
        self.fc2 = nn.Linear(fc1_units, fc2_units)
        self.fc3 = nn.Linear(fc2_units, action_size)
        self.reset_parameters()

    def reset_parameters(self):
        self.fc1.weight.data.uniform_(*hidden_init(self.fc1))
        self.fc2.weight.data.uniform_(*hidden_init(self.fc2))
        self.fc3.weight.data.uniform_(-3e-3, 3e-3)

    def forward(self, state):
        """Build an actor (policy) network that maps states -> actions."""
        x = self.bn1(F.leaky_relu(self.fc1(state)))
        x = F.leaky_relu(self.fc2(x))
        return F.tanh(self.fc3(x))
```

<u>Architecture of the *Critor* net.</u>

```
        self.fcs1 = nn.Linear(state_size, fcs1_units)
        self.bn1 = nn.BatchNorm1d(fcs1_units)
        self.fc2 = nn.Linear(fcs1_units+action_size, fc2_units)
        self.fc3 = nn.Linear(fc2_units, 1)
        self.reset_parameters()

    def reset_parameters(self):
        self.fcs1.weight.data.uniform_(*hidden_init(self.fcs1))
        self.fc2.weight.data.uniform_(*hidden_init(self.fc2))
        self.fc3.weight.data.uniform_(-3e-3, 3e-3)

    def forward(self, state, action):
        """Build a critic (value) network that maps (state, action) pairs -> Q-values."""
        xs = self.bn1(F.leaky_relu(self.fcs1(state)))
        x = torch.cat((xs, action), dim=1)
        x = F.leaky_relu(self.fc2(x))
        return self.fc3(x)
```

2. Parameter set up and preliminary results

Table 1. shows the values for the parameters above.

Param.	value	Param.	value
BUFFER_SIZE	10^6	**LR(actor&critor)**	10^{-3}

BATCH_SIZE	128	**UPDATE_EVERY**	20
GAMMA	0.99	**TAU**	10^{-3}
NUM_LEARNING	10	**EPSILON_DECAY**	10^{-6}
WEIGHT_DECAY	0.		

Table 1 Parameters

Figure 2. plots the rewards per episode.

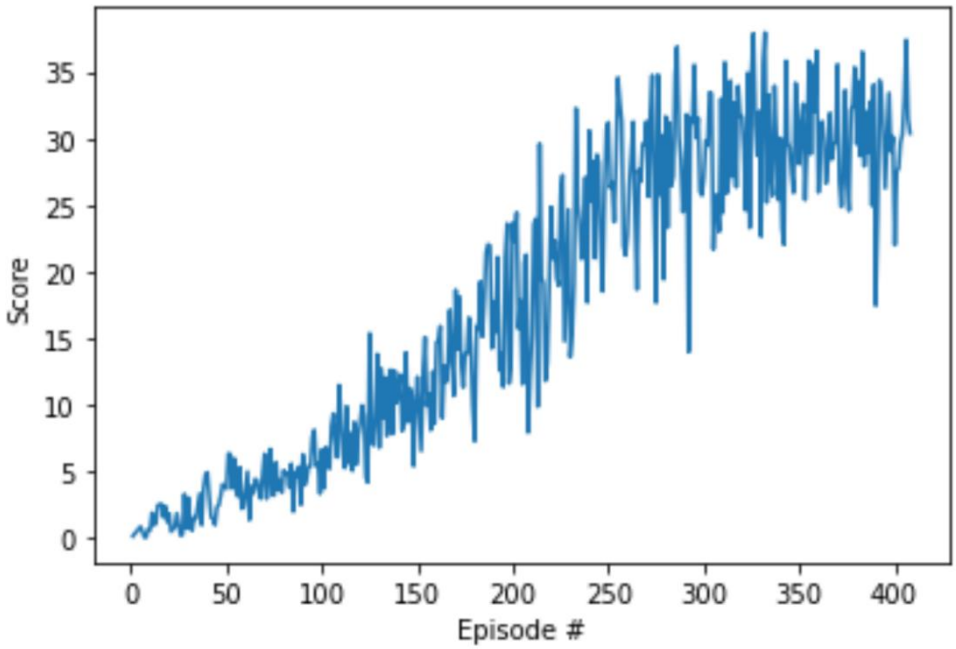

Figure 2 rewards per episode

Table 2. lists the average reward per 100 episodes, and the agent is able to receive an average reward (over 100 episodes) of 30 in 989 episodes.

```
Episode 10      Average Score: 0.38     Score: 0.53
Episode 20      Average Score: 1.08     Score: 1.76
Episode 30      Average Score: 1.12     Score: 2.97
Episode 40      Average Score: 1.44     Score: 4.87
Episode 50      Average Score: 1.67     Score: 3.65
Episode 60      Average Score: 2.10     Score: 3.53
Episode 70      Average Score: 2.37     Score: 6.27
Episode 80      Average Score: 2.60     Score: 5.03
Episode 90      Average Score: 2.80     Score: 6.23
Episode 100     Average Score: 3.08     Score: 6.60
Episode 110     Average Score: 3.77     Score: 9.253
Episode 120     Average Score: 4.29     Score: 8.55
Episode 130     Average Score: 5.04     Score: 6.738
Episode 140     Average Score: 5.84     Score: 11.04
Episode 150     Average Score: 6.59     Score: 12.06
Episode 160     Average Score: 7.24     Score: 14.67
Episode 170     Average Score: 8.22     Score: 18.60
Episode 180     Average Score: 9.11     Score: 7.233
Episode 190     Average Score: 10.48    Score: 17.74
Episode 200     Average Score: 11.60    Score: 23.64
Episode 210     Average Score: 12.55    Score: 15.17
Episode 220     Average Score: 13.79    Score: 24.84
Episode 230     Average Score: 14.99    Score: 13.55
Episode 240     Average Score: 16.29    Score: 30.58
Episode 250     Average Score: 17.88    Score: 31.20
Episode 260     Average Score: 19.50    Score: 22.77
Episode 270     Average Score: 20.87    Score: 31.29
Episode 280     Average Score: 22.32    Score: 31.65
Episode 290     Average Score: 23.40    Score: 26.28
Episode 300     Average Score: 24.57    Score: 27.03
Episode 310     Average Score: 25.68    Score: 24.41
Episode 320     Average Score: 26.80    Score: 31.63
Episode 330     Average Score: 27.72    Score: 22.62
Episode 340     Average Score: 28.43    Score: 30.03
Episode 350     Average Score: 28.71    Score: 28.06
Episode 360     Average Score: 29.14    Score: 25.94
Episode 370     Average Score: 29.42    Score: 35.55
Episode 380     Average Score: 29.58    Score: 29.57
Episode 390     Average Score: 29.66    Score: 17.43
Episode 400     Average Score: 29.75    Score: 21.98
Episode 408     Average Score: 30.01    Score: 30.38
Environment solved in 408 Episodes      Average Score: 30.01
```

Table 2 average rewards per 100 episodes

3. Tendencies that might improve the performance

Observation1. In the training, we find that the nodes (512,256) model performs a bit better than the nodes (256,128). In average, the nodes (512,256) can achieve around 26 scores while the nodes (256,128) achieve 24. A straight forward tendency is to add more layers for a better performance in the future work.

Observation2. We have observed the effects of whether to use epsilon decay for adding the noise, whether to use the OU noise or random noise, and whether we should reset the OU noise parameters after each learning step. The results are given below:

reset_noise	epsilon_decay	random_noise	score after 1000 episodes
False	True	True	3.4
False	False	True	12.84
False	True	False	3.07
True	True	False	8.99
False	False	False	5.34
True	False	False	8.32

We can see that at least in this case, the use of OU process is not very necessary. Meanwhile, the use of epsilon decay for the OU noise is also not very necessary. However, we do expect to use a noise reset after each learning step. This indicates that the use of these parameters needs to be checked and compared for a new learning.

Observation3. When we set the L2 WEIGHT_DECAY to zero, the performance has been significantly improved. Therefore, in the case we should not use L2 WEIGHT_DECAY. This indicates that new architectures that take into account the physical laws should be investigated.